

LAPORAN TUGAS PRAKTIKUM
STRUKTUR DATA
PERTEMUAN 11



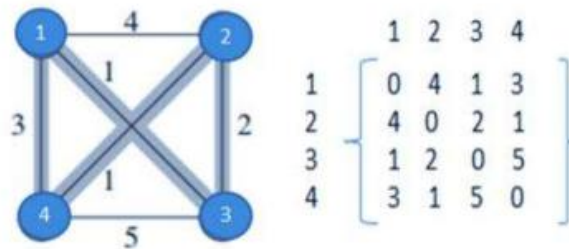
Nama : Jaw Syanito Riptanto Putra
NPM : 2510631170040
Kelas : 2 A

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG

2025

SOAL

LATIHAN:



Di atas kita dapat melihat grafik dan matriks biaya yang lengkap yang mencakup jarak antara masing-masing desa. Kita dapat mengamati bahwa matriks biaya adalah simetris yang berarti jarak antara desa 2 hingga 3 sama dengan jarak antara desa 3 hingga 2. Masalah di sini adalah penjual keliling ingin mengetahui turnya dengan biaya minimum. Katakanlah Tukang post berada dititik 1 dia berkeliling semua titik lainnya yang belum dikunjungi dan kembali ke titik awal dengan biaya minimum

Buatlah program dengan c++ dengan mengimplementasikan adjacency matriks dan beberapa algoritma untuk menemukan solusi dari Tukang Pos, untuk titik awal berupa inputnya

Contoh Outputnya

Cost List :

0	4	1	3
4	0	2	1
1	2	0	5
3	1	5	0

Jalur Terpendek :

1→3→2→4→1

Minimum Cost : 7

JAWABAN

Penjelasan :

Saya membuat program sederhana yang berfungsi untuk mencari jalur perjalanan paling pendek menggunakan konsep struktur data Graph dan algoritma Traveling Salesman Problem (TSP). Program ini menggambarkan seorang tukang pos yang harus mengunjungi seluruh desa satu kali dan kembali ke desa awal dengan total jarak seminimal mungkin. Program menggunakan matriks adjacency untuk menyimpan jarak antar desa serta menerapkan rekursi dan backtracking untuk mencoba seluruh kemungkinan jalur perjalanan.

Di bagian awal program, saya menambahkan `#include <iostream>` agar program dapat menggunakan perintah input dan output seperti `cin` dan `cout`. Kemudian saya menggunakan `using namespace std;` supaya penulisan fungsi standar C++ tidak perlu diawali dengan `std::`.

Selanjutnya saya membuat sebuah konstanta bernama `MAX` dengan nilai 10. Konstanta ini digunakan sebagai batas maksimum jumlah desa atau ukuran array yang dipakai dalam program. Dengan adanya konstanta ini, ukuran array dapat digunakan secara konsisten di seluruh program.

Setelah itu saya mendeklarasikan variabel `n` dengan nilai 4 yang menunjukkan jumlah desa atau titik pada graph. Artinya program akan memproses empat buah desa yang saling terhubung.

Kemudian saya membuat sebuah matriks dua dimensi bernama `jarakDesa`. Matriks ini digunakan untuk menyimpan jarak antar desa dalam bentuk adjacency matrix. Setiap baris dan kolom merepresentasikan desa tertentu. Misalnya nilai 4 pada baris pertama kolom kedua berarti jarak dari desa pertama ke desa kedua adalah 4. Nilai 0 pada diagonal matriks menunjukkan bahwa jarak dari sebuah desa ke dirinya sendiri adalah nol.

Selanjutnya saya mendeklarasikan array boolean bernama `diKunjungi`. Array ini digunakan untuk menandai apakah suatu desa sudah dikunjungi atau belum selama proses pencarian jalur. Jika bernilai `true`, berarti desa tersebut sudah dikunjungi, sedangkan jika bernilai `false`, berarti desa masih belum dikunjungi.

Kemudian saya membuat dua buah array bernama `bestPath` dan `tempPath`. Array `tempPath` digunakan untuk menyimpan jalur sementara yang sedang dicoba oleh program selama proses rekursi berlangsung. Sedangkan `bestPath` digunakan untuk menyimpan jalur terbaik yang memiliki total jarak paling kecil.

Setelah itu saya mendeklarasikan variabel `jarakMinimal`. Variabel ini digunakan untuk menyimpan total jarak terkecil yang ditemukan selama proses pencarian jalur. Nantinya setiap jalur yang ditemukan akan dibandingkan dengan nilai ini.

Selanjutnya saya membuat sebuah fungsi rekursif bernama `tsp` yang merupakan inti dari algoritma Traveling Salesman Problem. Fungsi ini memiliki parameter `current` sebagai posisi desa saat ini, `start` sebagai desa awal, `count` sebagai jumlah desa yang sudah dikunjungi, dan `jarakTotal` sebagai total jarak perjalanan sementara.

Di dalam fungsi tersebut, program pertama kali melakukan pengecekan menggunakan kondisi `if (count == n)`. Kondisi ini digunakan untuk mengetahui apakah seluruh desa sudah dikunjungi. Jika semua desa telah dikunjungi, maka program akan menambahkan jarak dari desa terakhir kembali ke desa awal agar perjalanan membentuk siklus lengkap.

Kemudian program menyimpan desa awal ke dalam `tempPath[count]` sebagai penutup jalur perjalanan. Setelah itu dilakukan pengecekan apakah `jarakTotal` lebih kecil dibandingkan `jarakMinimal`. Jika lebih kecil, berarti jalur yang ditemukan merupakan jalur terbaik sementara.

Jika ditemukan jalur yang lebih pendek, maka nilai `jarakMinimal` diperbarui dengan total jarak terbaru. Selanjutnya program menggunakan perulangan `for` untuk menyalin seluruh isi `tempPath` ke dalam `bestPath` agar jalur terbaik dapat disimpan.

Jika semua desa belum dikunjungi, maka program masuk ke bagian pencarian node berikutnya menggunakan perulangan `for`. Perulangan ini digunakan untuk mencoba seluruh kemungkinan desa tujuan berikutnya.

Di dalam perulangan tersebut, program melakukan pengecekan menggunakan kondisi `if (!diKunjungi[i] && jarakDesa[current][i] != 0)`. Kondisi ini memastikan bahwa desa tujuan belum pernah dikunjungi dan memiliki jalur yang terhubung dengan desa saat ini.

Jika kondisi terpenuhi, maka desa tersebut ditandai sebagai sudah dikunjungi dengan `diKunjungi[i] = true`. Kemudian indeks desa disimpan ke dalam `tempPath[count]` sebagai bagian dari jalur sementara.

Setelah itu program memanggil kembali fungsi `tsp` secara rekursif dengan parameter desa tujuan yang baru, jumlah desa yang sudah dikunjungi ditambah satu, serta total jarak yang diperbarui dengan menambahkan jarak antar desa.

Ketika proses rekursi selesai, program melakukan backtracking dengan mengubah kembali nilai `diKunjungi[i]` menjadi `false`. Langkah ini penting agar desa tersebut dapat dicoba kembali pada kombinasi jalur yang lain.

Selanjutnya program masuk ke fungsi utama `main()`. Di dalam fungsi ini saya mendeklarasikan variabel `start` yang digunakan untuk menyimpan titik awal perjalanan tukang pos.

Program kemudian menampilkan matriks jarak antar desa menggunakan perulangan bersarang `for`. Perulangan pertama digunakan untuk menampilkan baris matriks, sedangkan perulangan kedua digunakan untuk menampilkan isi setiap kolom matriks.

Setelah matriks ditampilkan, program meminta pengguna memasukkan titik awal perjalanan dengan perintah `cin >> start`. Karena indeks array dimulai dari nol, maka nilai `start` dikurangi satu menggunakan `start--`.

Kemudian desa awal ditandai sebagai sudah dikunjungi dengan `diKunjungi[start] = true`. Selanjutnya posisi desa awal disimpan pada `tempPath[0]`.

Program lalu memanggil fungsi `tsp(start, start, 1, 0)` untuk memulai proses pencarian jalur terpendek. Parameter pertama menunjukkan posisi awal, parameter kedua menunjukkan desa awal tujuan kembali, parameter ketiga menunjukkan jumlah desa yang sudah dikunjungi, dan parameter keempat menunjukkan total jarak awal yang masih nol.

Setelah proses pencarian selesai, program menampilkan tulisan "Jalur Terpendek" lalu menggunakan perulangan `for` untuk mencetak isi array `bestPath`. Setiap indeks ditampilkan dalam bentuk nomor desa dengan menambahkan angka satu agar sesuai dengan nomor desa yang dipahami pengguna.

Di antara setiap desa, program menampilkan tanda panah " ---> " untuk memperjelas urutan perjalanan yang harus ditempuh.

Terakhir program menampilkan total jarak paling pendek yang tersimpan di dalam variabel `jarakMinimal`. Setelah seluruh proses selesai, program mengembalikan nilai 0 sebagai tanda bahwa program berhasil dijalankan dengan baik.

Kode Programnya :

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAX = 10;
5
6  //Jumlah titik / desa
7  int n = 4;
8
9  //Matriks sesuai soal (Jarak antar desa)
10 int jarakDesa[MAX][MAX] = {
11     {0, 4, 1, 3},
12     {4, 0, 2, 1},
13     {1, 2, 0, 5},
14     {3, 1, 5, 0}
15 };
16
17 // Status desa (dikunjungi atau belum)
18 bool dikunjungi[MAX];
19
20 // Menyimpan jalur terbaik
21 int bestPath[MAX + 1];
22 int tempPath[MAX + 1];
23
24 // Nilai awal minimum jarak
25 int jarakMinimal = 999999;
26
27 // Fungsi TSP (Traveling Salesman Problem) pakai rekursif
28 void tsp(int current, int start, int count, int jarakTotal) {
29
30     // Jika semua node sudah dikunjungi
31     if (count == n) {
32
33         jarakTotal += jarakDesa[current][start];
34         tempPath[count] = start;
35
36         // Cek jarak minimalnya
37         if (jarakTotal < jarakMinimal) {
38             jarakMinimal = jarakTotal;
39
40             // Simpan jalur terbaik
41             for (int i = 0; i <= n; i++) {
42                 bestPath[i] = tempPath[i];
43             }
44         }
45         return;
46     }
47 }
```

```

48 // Mengunjungi node lain
49 for (int i = 0; i < n; i++) {
50
51     if (!dikunjungi[i] && jarakDesa[current][i] != 0) {
52
53         dikunjungi[i] = true;
54         tempPath[count] = i;
55
56         tsp(i, start, count + 1,
57             jarakTotal + jarakDesa[current][i]);
58
59         dikunjungi[i] = false;
60     }
61 }
62 }
63
64 int main() {
65     int start;
66     cout << "Jarak antar desa : \n";
67     for (int i = 0; i < n; i++) {
68
69         for (int j = 0; j < n; j++) {
70             cout << jarakDesa[i][j] << "\t";
71         }
72         cout << endl;
73     }
74
75     //Titik awal tukang pos
76     cout << "\nMasukkan titik awal (1-4): ";
77     cin >> start;
78     start--;
79
80     dikunjungi[start] = true;
81     tempPath[0] = start;
82
83     tsp(start, start, 1, 0);
84
85     cout << "\nJalur Terpendek : \n";
86
87     for (int i = 0; i <= n; i++) {
88
89         cout << bestPath[i] + 1;
90
91         if (i != n) {
92             cout << " ---> ";
93         }
94     }
95
96     cout << "\n\nJarak terpendek : " << jarakMinimal;
97
98     return 0;
99 }

```

Outputnya di terminal :

```
Jarak antar desa :
0      4      1      3
4      0      2      1
1      2      0      5
3      1      5      0

Masukkan titik awal (1-4): 1

Jalur Terpendek :
1 ---> 3 ---> 2 ---> 4 ---> 1

Jarak terpendek : 7
```

Link Repository GITHUB :

https://github.com/jawsyanito/StrukturData_25040/tree/main/Petemuan%2011/Tugas